

Blockchains & Cryptocurrencies

Mechanics of Bitcoin



Instructor: Matthew Green
Johns Hopkins University - Spring 2026

Many slides based on NBFMG

Housekeeping

- Pace of material?
- Office hours: 2pm Weds
- A2b is due in the future
- **Project groups and proposal is due 2/25 end of day**

News?

Crypto Funds Post Fourth Straight Week Of Outflows As US Investors Exit



Kostiantyn Tsentsura 1 hour ago

#Bitcoin #Ripple #Ethereum



News?

Address Poisoning Scams Drain Millions as Crypto Attacks Surge

Crypto users lost tens of millions of dollars in January to address poisoning and signature phishing scams, as attackers exploited lower transaction costs and user inattention to steal funds at scale.

 Listen to article

 Make us preferred on Google

WRITTEN BY



EMMANUEL MUSA

SHARE



Bitcoin.com
News



100
Wa

CRY

100
Ca



130

Web3

Bitcoin needs a quantum upgrade. So why isn't it happening?



Some prominent Bitcoin developers
Shutterstock

Home > Web3 > Bitcoin needs a quantum upgrade. So why isn't it happening?

By Tim Craig

16 February 2026 at 03:09

- Bitcoin is running out of time to adopt post-quantum cryptography.

Today

- Finish talking about Bitcoin at a high level
- Talk about the low-level mechanics of real Bitcoin

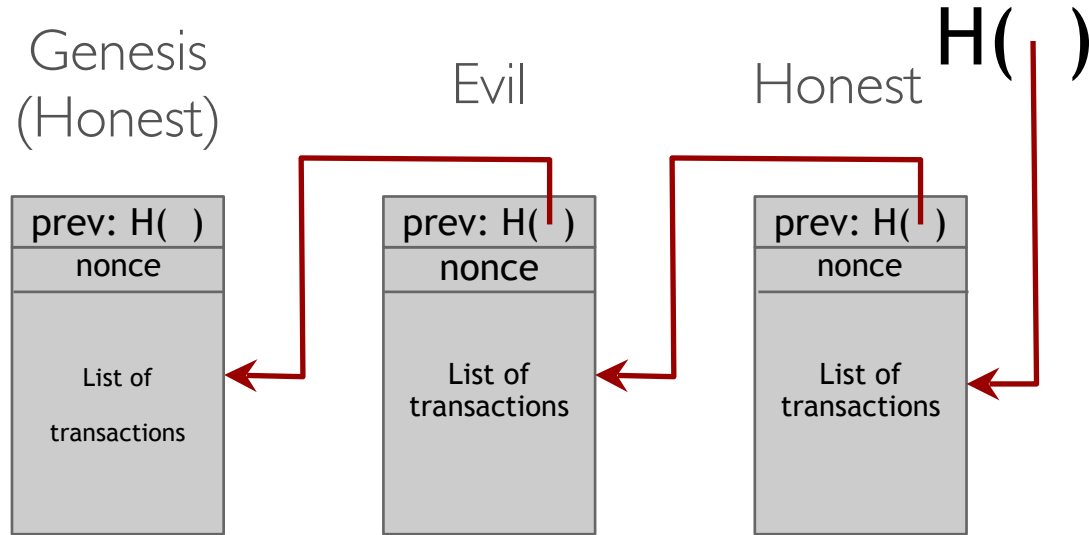
Bitcoin: A Peer-to-Peer Electronic Cash System

Satoshi Nakamoto
satoshin@gmx.com
www.bitcoin.org

Abstract. A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution. Digital signatures provide part of the solution, but the main benefits are lost if a trusted third party is still required to prevent double-spending. We propose a solution to the double-spending problem using a peer-to-peer network. The network timestamps transactions by hashing them into an ongoing chain of hash-based proof-of-work, forming a record that cannot be changed without redoing the proof-of-work. The longest chain not only serves as proof of the sequence of events witnessed, but proof that it came from the largest pool of CPU power. As long as a majority of CPU power is controlled by nodes that are not cooperating to attack the network, they'll generate the longest chain and outpace attackers. The network itself requires minimal structure. Messages are broadcast on a best effort basis, and nodes can leave and rejoin the network at will, accepting the longest proof-of-work chain as proof of what happened while they were gone.

1. Introduction

Review



Selecting T (Bitcoin)

Every block contains a (packed) encoding of T (target) which corresponds to “*difficulty*” (*d*)

$d = (2^{\{16\}} - 1) * 2^{\{8*26\}} / T$ <- *relationship between d, T*

Goals:

- Bitcoin block time should average 10 minutes
- 2016 blocks * 10 minutes == 2 weeks
- Everyone in the network agrees on T
- *Hence must be a function of chain data*

This brings back a notion of time

What if there's a collision?

Sometimes two separate nodes find a valid solution simultaneously

This can result in network partition



What if there's a collision?



Image CC-BY-3 Theymos taken from the Bitcoin wiki



This is good and bad

Good: if we experience a “chain fork” and the network is connected (i.e., not totally partitioned), then eventually we will learn about both forks

Good: if the “hash power” behind the two chains is unequal, we will probably end up with one chain getting longer

Even if the hash power is equal, the inherent randomness of the puzzle (PoW) will likely cause one chain to advance

As one chain grows longer, other nodes will adopt it, and start adding to it

Q: What if difficulty changes?

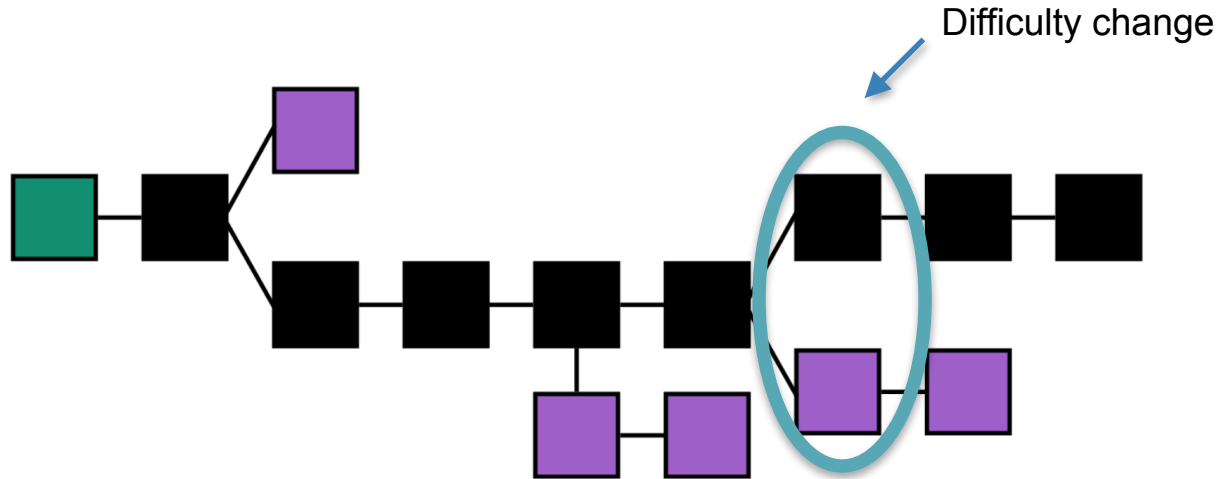


Image CC-BY-3 Theymos taken from the Bitcoin wiki

Q: What if difficulty changes?

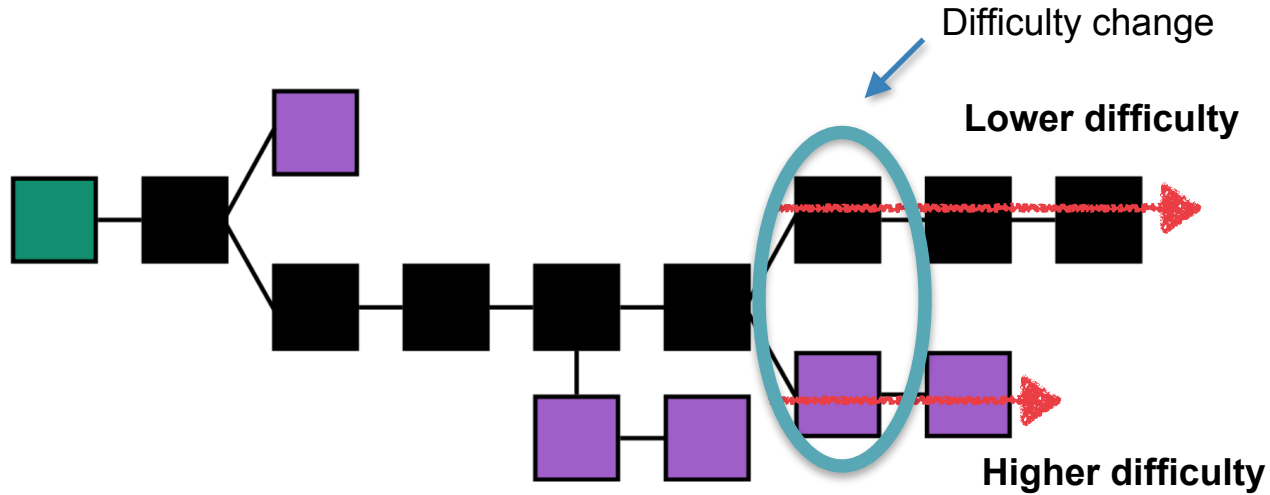


Image CC-BY-3 Theymos taken from the Bitcoin wiki

A: “Chain with most hashwork”

Bitcoin doesn't exactly use the longest chain rule

Instead, it employs a calculation that takes block difficulty into account

Each block has a difficulty. Convert to expected # of hashes to find block. Total these values. Chain with largest total is “longest”.

Most of the time, this is equivalent to longest chain

How many blocks can the adversary make?

Consider an adversary that controls a r -fraction of the hash power

How many blocks in expectation can they build in a t -block window?

What is the probability that they dominate that t -block window?

We will see some attacks that leverage these simplified calculations later....



How do we incentivize mining?



How do we incentivize mining?

Two answers to this question in Bitcoin:

1. “Transaction fees” Each transaction has a “tip” that can be collected by the node who mines it into a block (incentivizes inclusion of transactions)
2. “Block reward” 50/25/12.5/... BTC made from scratch (in a special Coinbase transaction) and given to the miner

Bitcoin transactions

An account-based ledger (*not* Bitcoin)

time



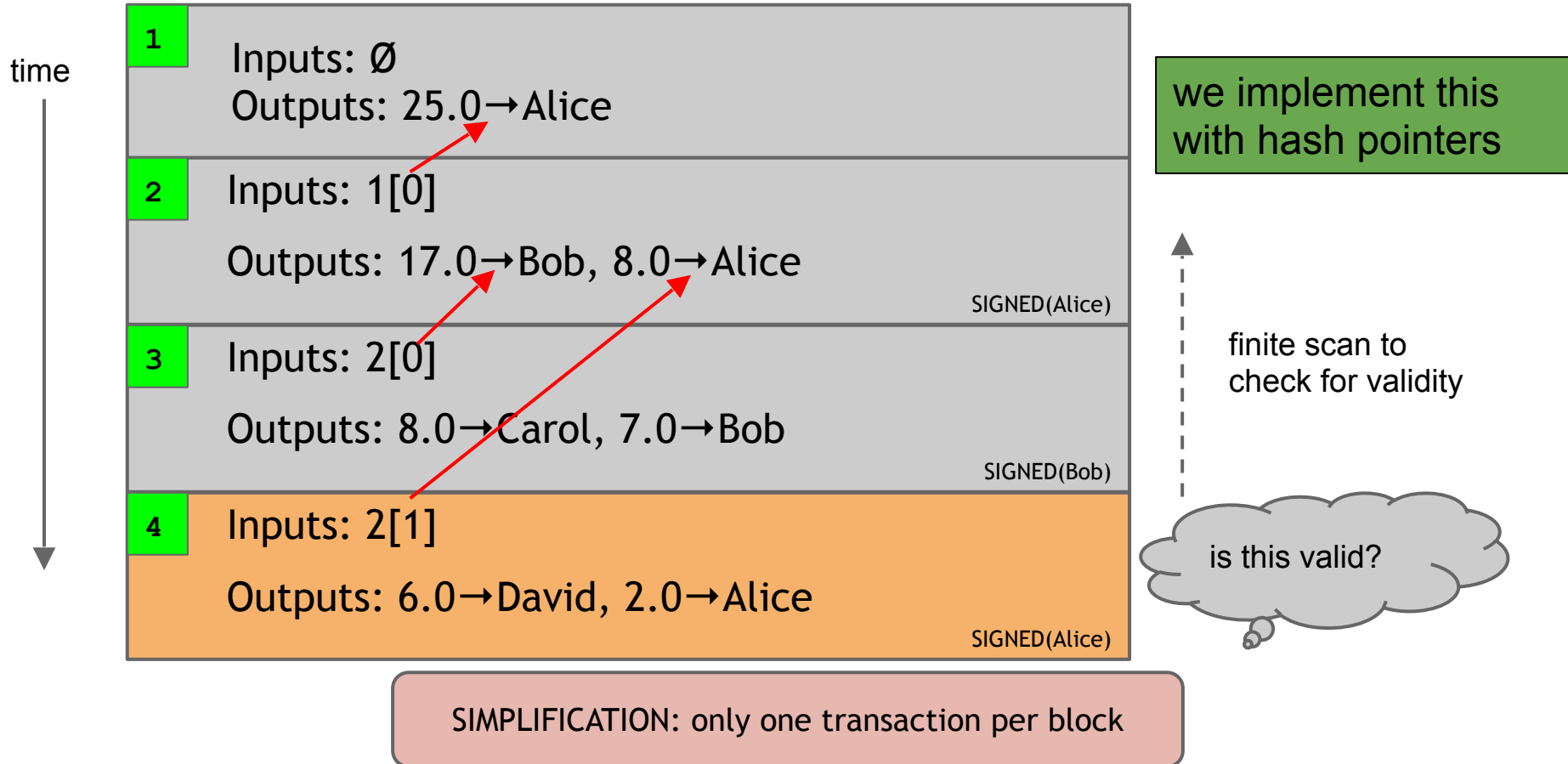
Create 25 coins and credit to Alice	ASSERTED BY MINERS
Transfer 17 coins from Alice to Bob	SIGNED(Alice)
Transfer 8 coins from Bob to Carol	SIGNED(Bob)
Transfer 5 coins from Carol to Alice	SIGNED(Carol)
Transfer 15 coins from Alice to David	SIGNED(Alice) ...

might need to
scan backwards
until genesis!

is this valid?

SIMPLIFICATION: only one transaction per block

A transaction-based ledger (Bitcoin)

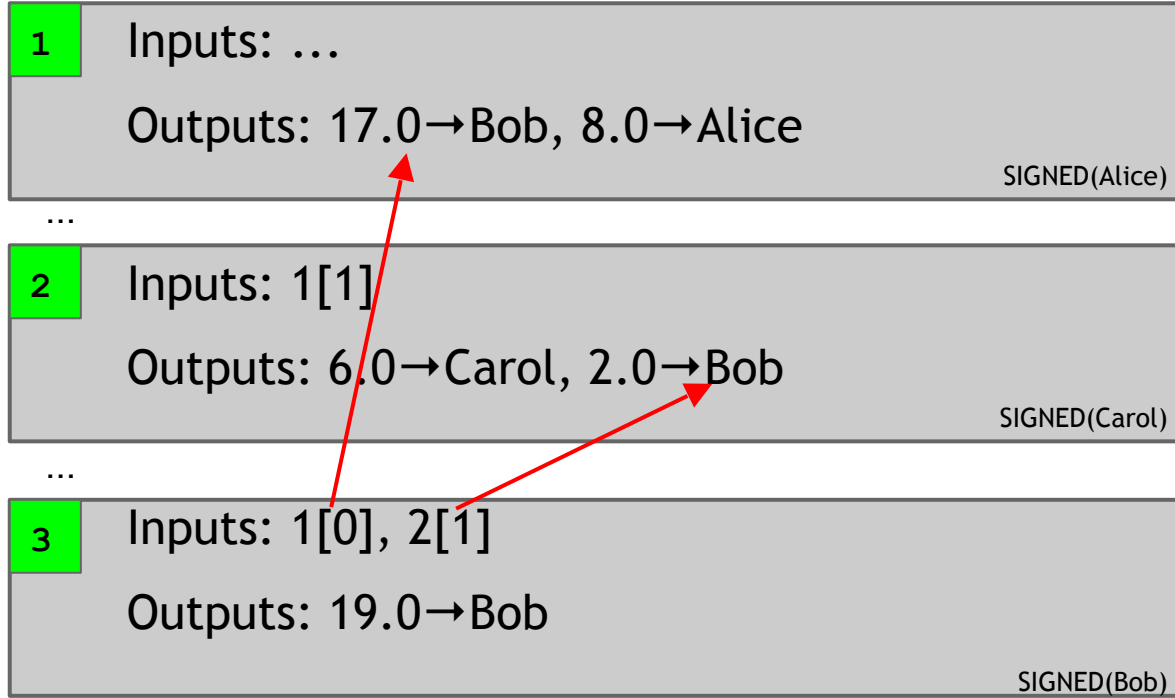


Referencing Transactions

- Hash pointers for transactions
- Within a transaction, refer to a particular output via serial numbers

Merging value

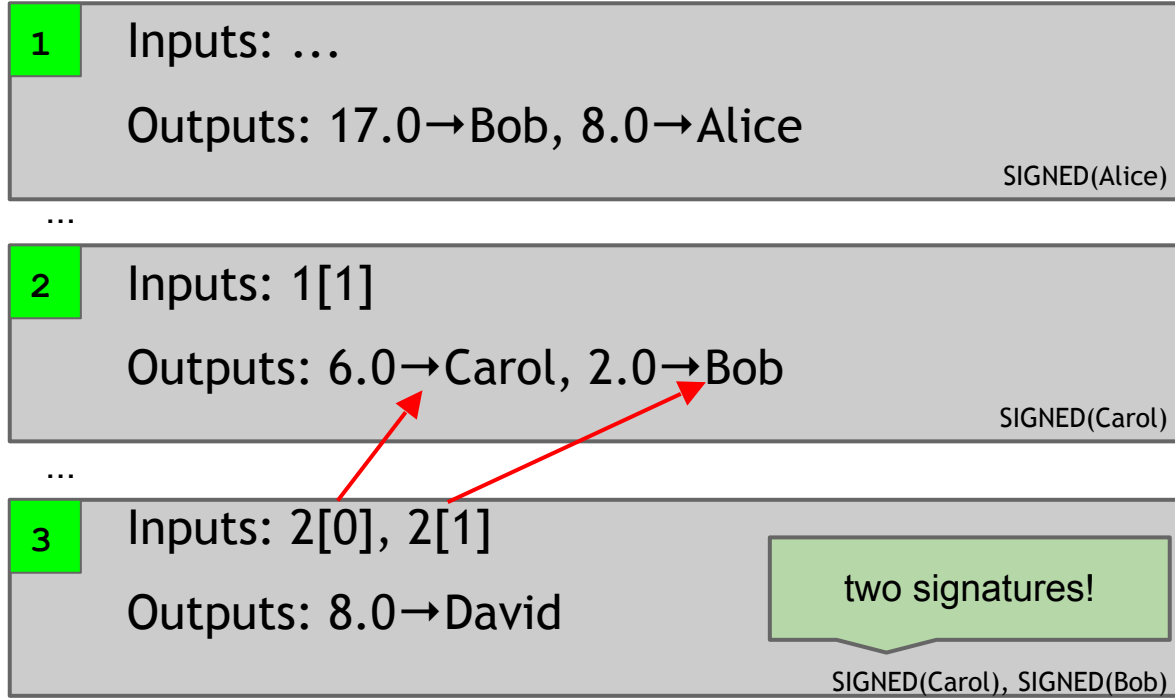
time



SIMPLIFICATION: only one transaction per block

Joint payments

time



SIMPLIFICATION: only one transaction per block

The real deal: a classical Bitcoin transaction

```
{  
  "hash": "5a42590fbe0a90ee8e8747244d6c84f0db1a3a24e8f1b95b10c9e050990b8b6b",  
  "ver": 1,  
  "vin_sz": 2,  
  "vout_sz": 1,  
  "lock_time": 0,  
  "size": 404,  
  "in": [  
    {  
      "prev_out": {  
        "hash": "3be4ac9728a0823cf5e2deb2e86fc0bd2aa503a91d307b42ba76117d79280260",  
        "n": 0  
      },  
      "scriptSig": "30440..."  
    },  
    {  
      "prev_out": {  
        "hash": "7508e6ab259b4df0fd5147bab0c949d81473db4518f81afc5c3f52f91ff6b34e",  
        "n": 0  
      },  
      "scriptSig": "3f3a4ce81...."  
    }  
  ],  
  "out": [  
    {  
      "value": "10.12287097",  
      "scriptPubKey": "OP_DUP OP_HASH160 69e02e18b5705a05dd6b28ed517716c894b3d42e OP_EQUALVERIFY OP_CHECKSIG"  
    }  
  ]  
}
```

metadata

input(s)

output(s)

The real deal: transaction metadata

{

transaction hash

{

"hash":"5a42590...b8b6b",

housekeeping

{

"ver":1,

"vin_sz":2,

"vout_sz":1,

"not valid before"

{

"lock_time":0,

housekeeping

{

"size":404,

more on this later...

...

}

The real deal: transaction inputs

```
"in":[
  {
    "prev_out":{
      "hash":"3be4...80260",
      "n":0
    },
    "scriptSig":"30440....3f3a4ce81"
  },
  ...
],
```

previous transaction {

signature {

(more inputs) {

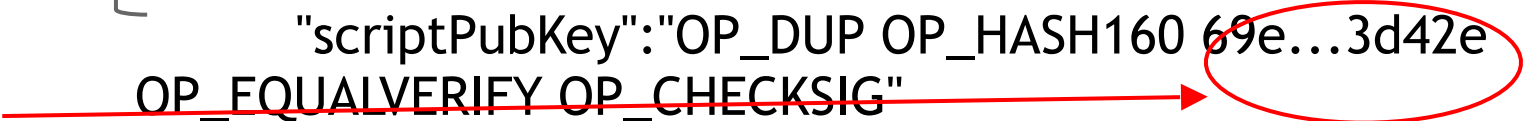
The real deal: transaction outputs

```
"out":[  
  {  
    "value":"10.12287097",  
    "scriptPubKey":"OP_DUP OP_HASH160 69e...3d42e  
OP_EQUALVERIFY OP_CHECKSIG"  
  },  
  ...  
],  
...  
(more outputs)
```

output value {

recipient address?? {

more on this soon...



Bitcoin scripts

Output “addresses” are really *scripts*

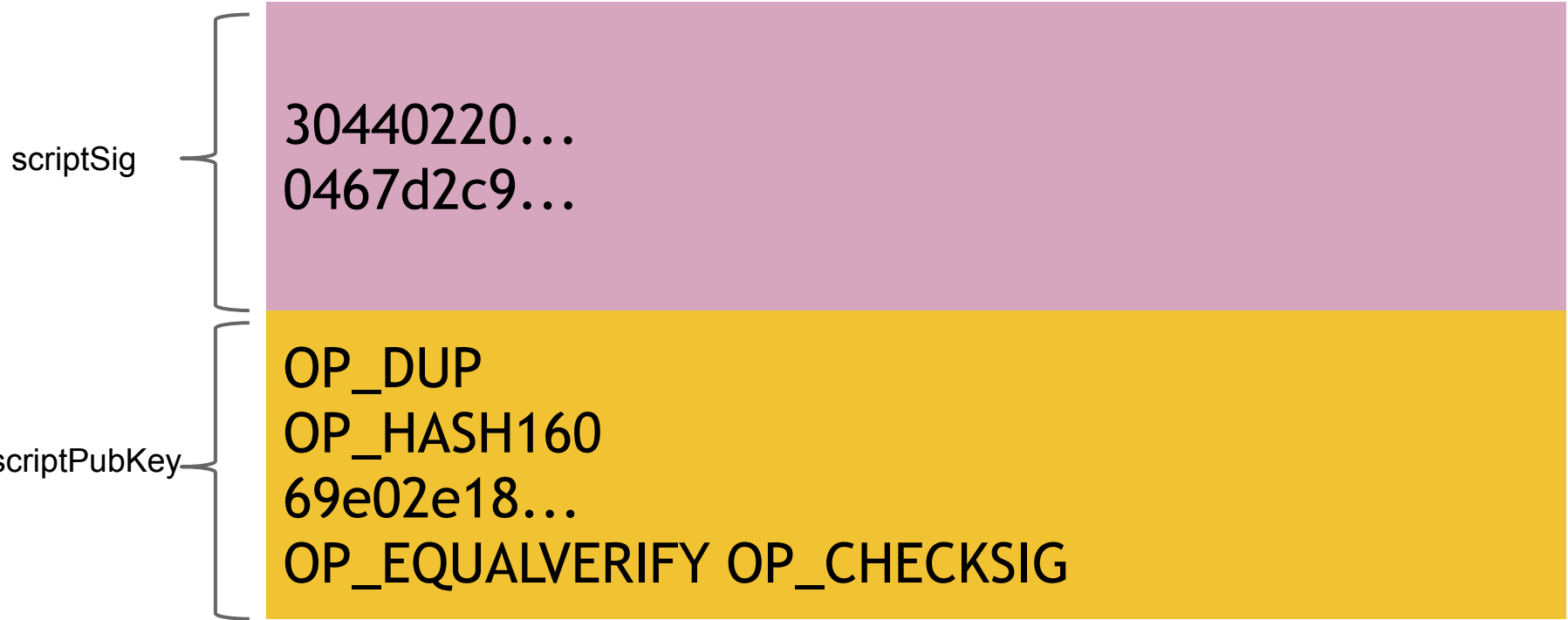
OP_DUP

OP_HASH160

69e02e18...

OP_EQUALVERIFY OP_CHECKSIG

Input “addresses” are *also* scripts



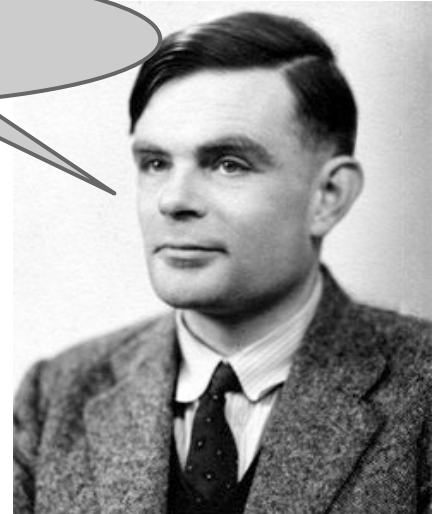
TO VERIFY: Concatenated script must execute completely with no errors

Bitcoin scripting language (“Script”)

Design goals

- Built for Bitcoin
- Simple, compact
- Support for cryptography
- Stack-based
- Limits on time/memory
- No looping

I am not
impressed



Bitcoin script execution example

<pubKeyHash?>
<pubKeyHash>
<pubKey>
true



<sig> <pubKey> OP_DUP OP_HASH160 <pubKeyHash?> OP_EQUALVERIFY OP_CHECKSIG

Bitcoin script instructions

256 opcodes total (15 disabled, 75 reserved)

- Arithmetic
- If/then
- Logic/data handling
- Crypto!
 - Hashes
 - Signature verification
 - Multi-signature verification

OP_CHECKMULTISIG

- Built-in support for joint signatures
- Specify n public keys
- Specify t
- Verification requires t signatures



BUG ALERT: Extra data value
popped from the stack and ignored

Bitcoin scripts in practice (“original”)

- Most nodes whitelist known scripts
- 99.9% are simple signature checks
- ~0.01% are MULTISIG
- ~0.01% are **Pay-to-Script-Hash**
- Remainder are errors, proof-of-burn

More on this soon

* numbers from NBFMG and slightly out of date

Proof-of-burn

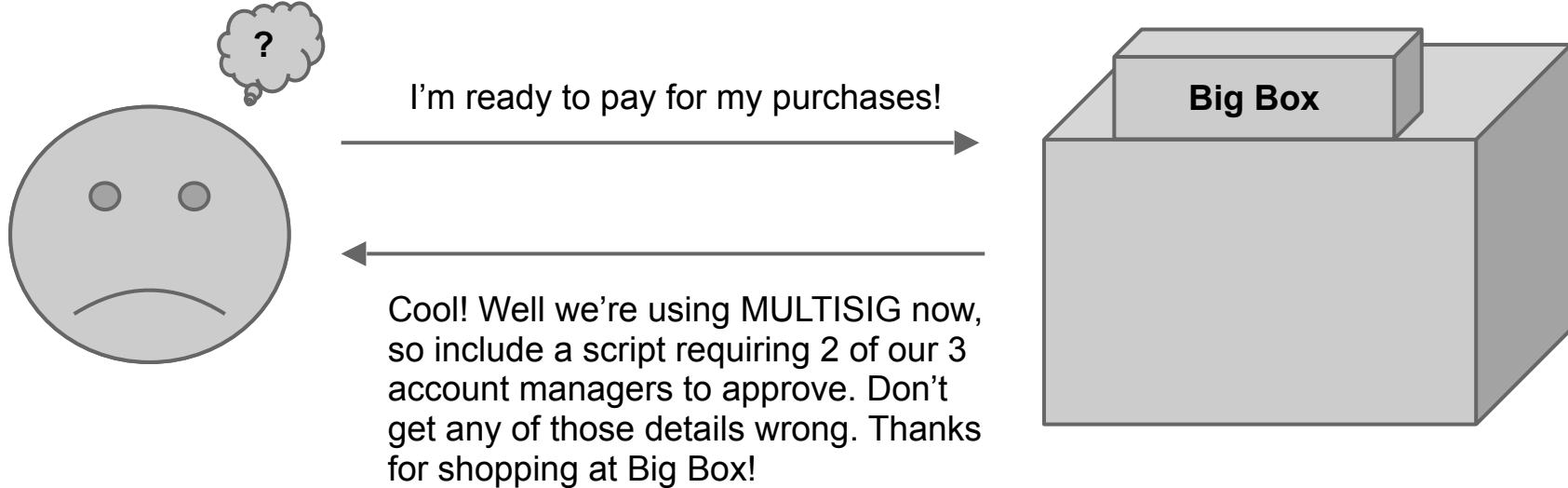
nothing's going to redeem that ☹️

OP_RETURN
<arbitrary data>

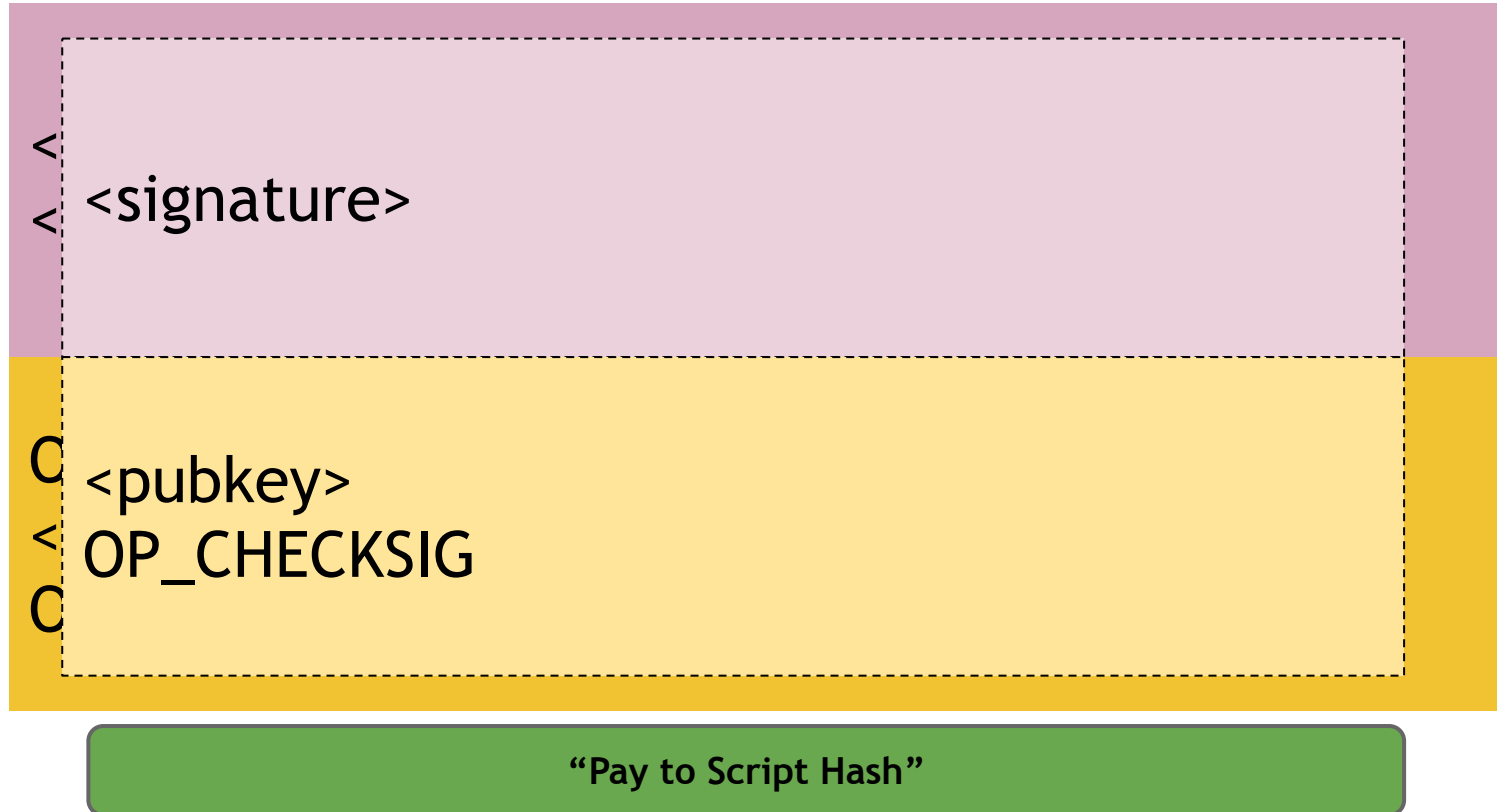
Proof-of-burn: Applications

- Can be used to publish arbitrary data on the blockchain (e.g., timestamping a document)
- Bootstrap Altcoins by requiring people to destroy bitcoins in order to get new altcoins

Should senders specify scripts?



Idea: use the hash of redemption script



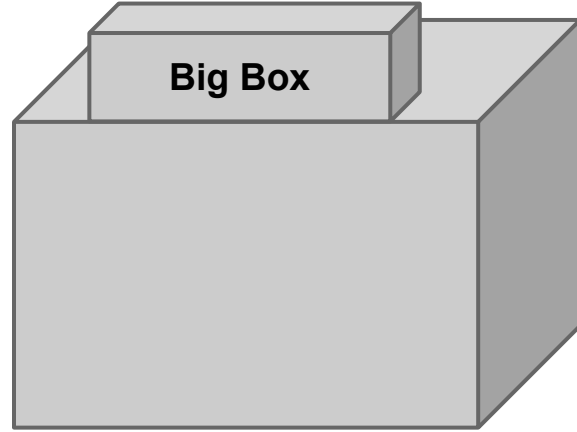
Pay to script hash



I'm ready to pay for my purchases!



Great! Here's our address: 0x3454



Block size limits and Segwit

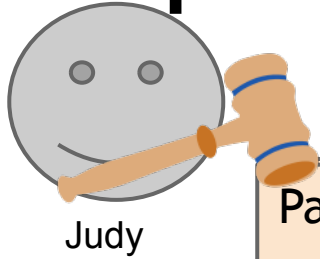
Applications of Bitcoin scripts

Example 1: “Fair” transactions

- Problem: Alice wants to buy a product from an online vendor Bob
- Alice doesn't want to pay until after Bob ships
- Bob doesn't want to ship until after Alice pays

Example 1: Fair transactions via Escrow

(disputed cases)



Pay x to Alice

SIGNED(ALICE, JUDY)



Alice

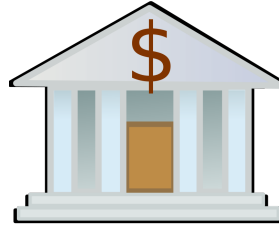
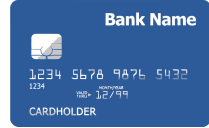


Bob

Pay x to 2-of-3 of Alice, Bob, Judy (MULTISIG)

SIGNED(ALICE)

Example 2: Green addresses



Bank

004 days since last double spend!

Faraday cage

It's me, Alice! Could you make out a green payment to Bob?

Pay x to Bob, y to Bank

No double spend

SIGNED(BANK)

Alice

PROBLEM: Alice wants to pay Bob.
Bob can't wait 6 verifications to guard against double-spends, or is offline completely.

Bob



Example 2: Micro-payments

- Pay-as-you-go WIFI: Alice wants to pay WIFI provider (Bob) for each minute of WIFI service. But she doesn't want to incur a transaction fee for every minute
- Similarly, pay-as-you-go online subscriptions
- Ad-free websites

Example 2: Micro-payments with Bitcoin

- Main Idea: Instead of doing several transactions, do a single transaction for total payment (and thus incur only a single transaction fee)
- *How to implement it?*

Example 2: Micro-payments with Bitcoin

What if Bob never signs??

all of these could
be double-
spends!

Input: x; Pay 42 to Bob, 58 to Alice

SIGNED(ALICE) SIGNED(BOB)

...

Alice demands a timed refund transaction before starting

Input: x; Pay 100 to Alice, LOCK until time t

SIGNED(ALICE) SIGNED(BOB)

I'm done!

Input: x; Pay 03 to Bob, 97 to Alice

SIGNED(ALICE) _____

I'll publish!

Input: x; Pay 02 to Bob, 98 to Alice

SIGNED(ALICE) _____

Input: x; Pay 01 to Bob, 99 to Alice

SIGNED(ALICE) _____

Input: y; Pay 100 to Bob/Alice (MULTISIG)

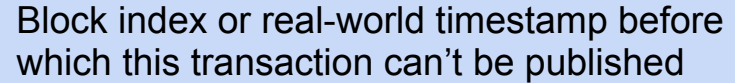
SIGNED(ALICE)

Alice

Bob

lock_time

```
{  
  "hash":"5a42590...b8b6b",  
  "ver":1,  
  "vin_sz":2,  
  "vout_sz":1,  
  "lock_time":315415,  
  "size":404,  
  ...  
}
```



Block index or real-world timestamp before
which this transaction can't be published

Micro-payments from Cryptocurrencies

More recent constructions, that achieve better properties

- Pass, shelat [CCS'16]
- Chiesa, Green, Liu, Miao, Miers, Mishra [EUROCRYPT'17]

More advanced scripts

- Fair multiplayer lotteries and fair multiparty computation [Andrychowicz-Dziembowski-Malinowski-Mazurek, S&P'14; Bentov-Kumaresan, CRYPTO'14]
- Hash pre-image challenges

“Smart contracts”

Later: More powerful smart contracts with Ethereum
(Turing-complete scripting language)

Bitcoin blocks

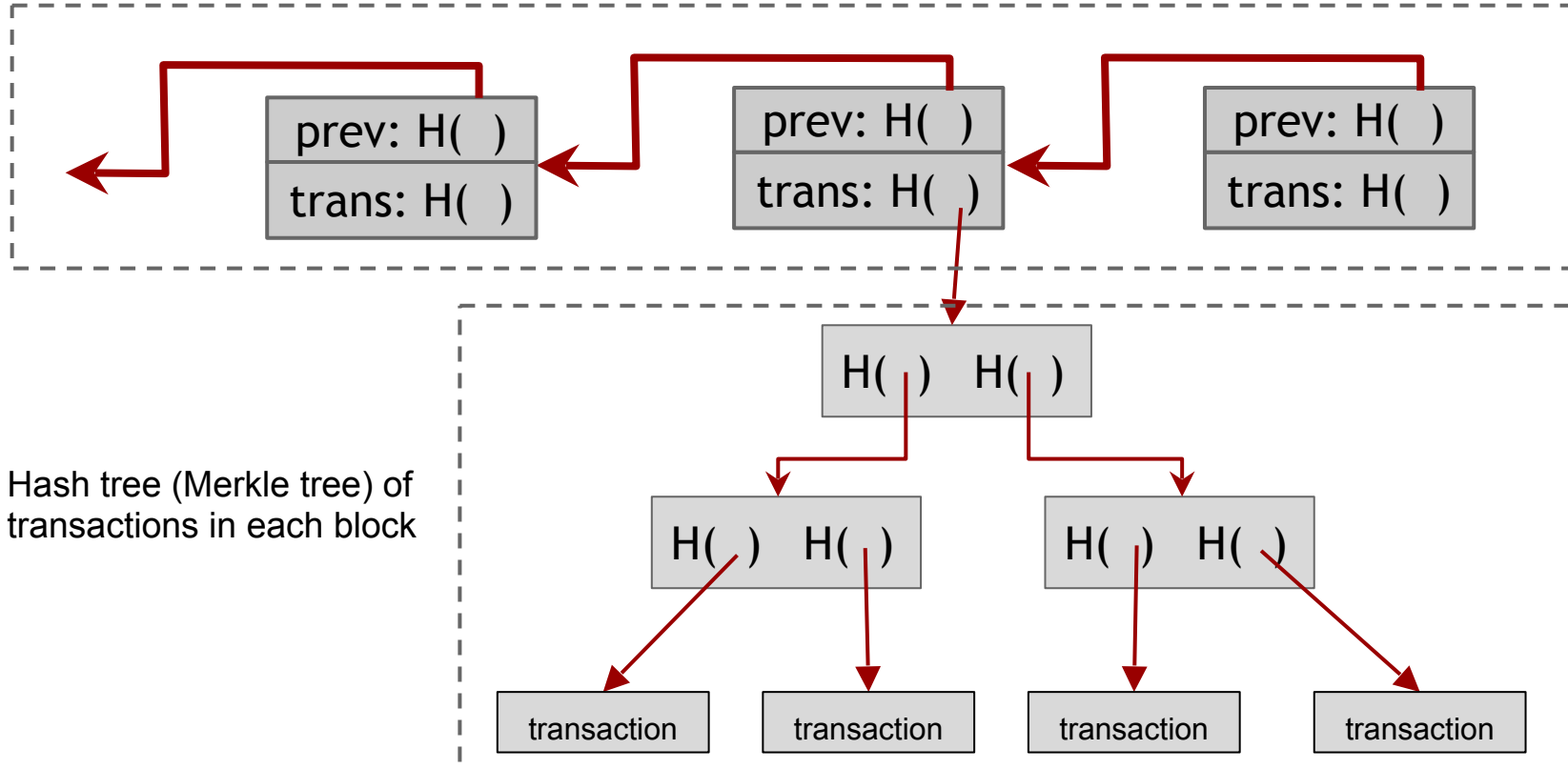
Bitcoin blocks

Why bundle transactions together?

- Single unit of work for miners
- Limit length of hash-chain of blocks
 - Faster to verify history

Bitcoin block structure

Hash chain of blocks

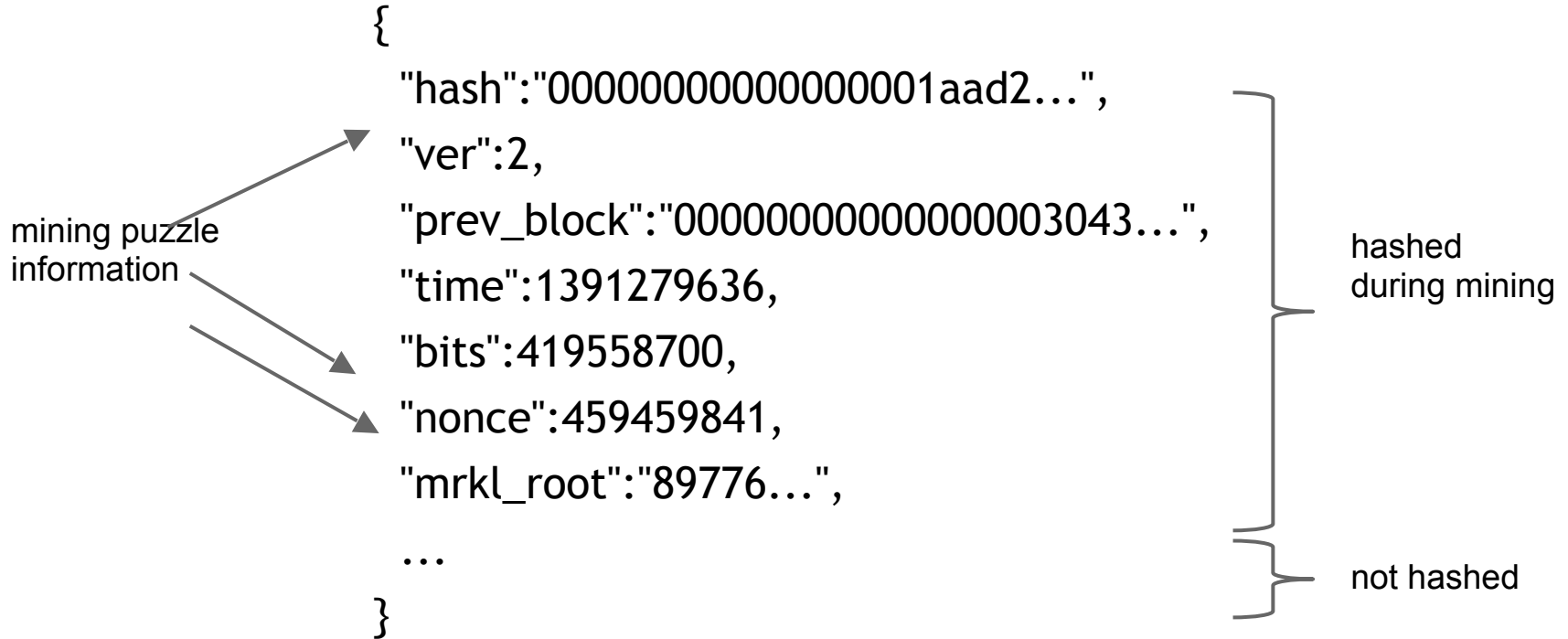


The real deal: a Bitcoin block

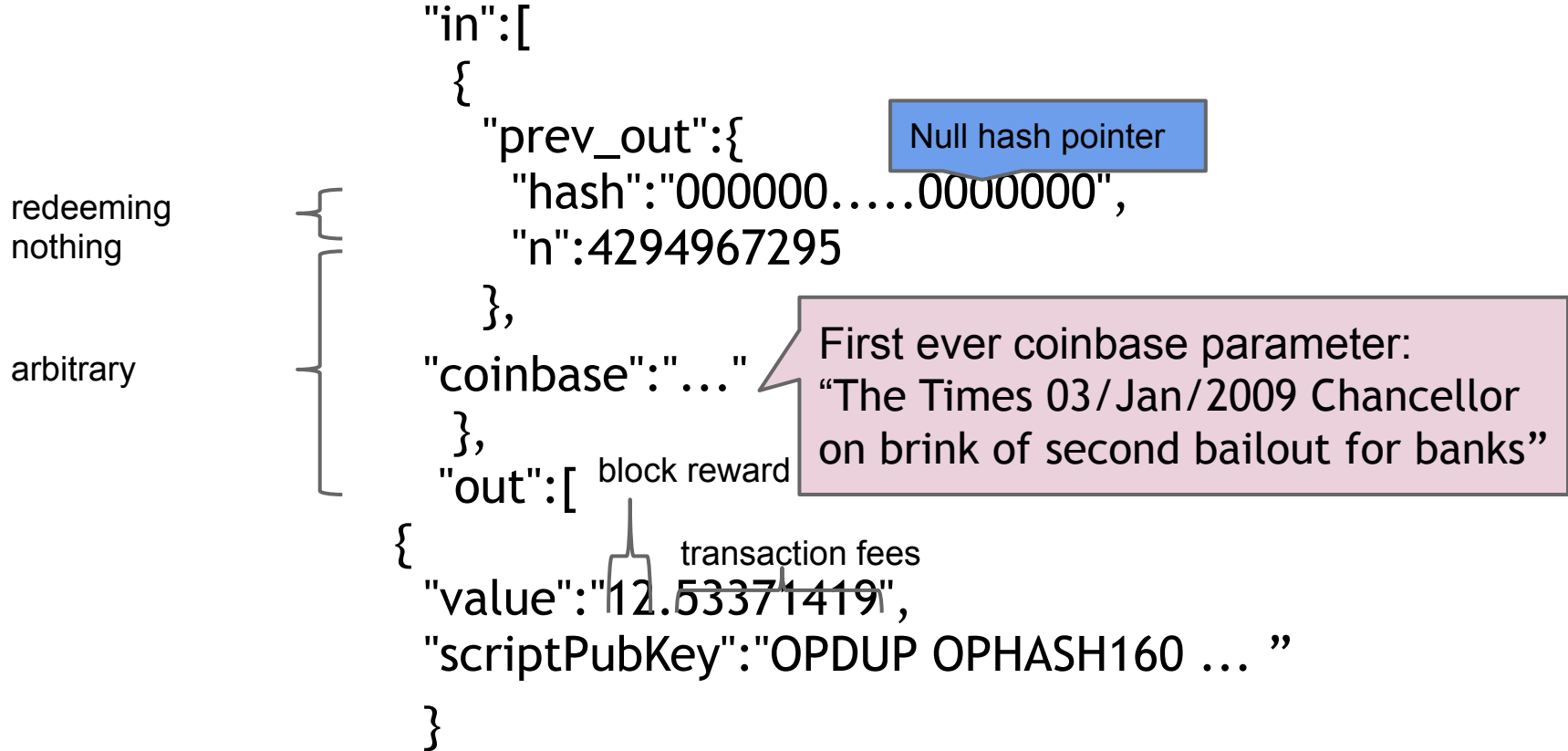
block header {
 "hash": "000000000000000001aad2...",
 "ver": 2,
 "prev_block": "00000000000000003043...",
 "time": 1391279636,
 "bits": 419558700,
 "nonce": 459459841,
 "mrkl_root": "89776...",
 "n_tx": 354,
 "size": 181520,
 "tx": [
 ...
],
 "mrkl_tree": [
 "6bd5eb25...",
 ...
 "89776cdb..."
]
}

transaction data

The real deal: a Bitcoin block header



The real deal: coinbase transaction



See for yourself!

Transaction View information about a bitcoin transaction

151b750d1f13e76d84e82b34b12688811b23a8e3119a1cba4b4810f9b0ef408d

1KryFUt9tXHvaoCYTNPbqpWPJKQ717YmL5



1KvdrQ3oGqMAiDTMEYCcdDSnVaGNW2YZh
1KryFUt9tXHvaoCYTNPbqpWPJKQ717YmL5

1.0194 BTC
3.458 BTC

9 Confirmations

4.4774 BTC

Summary

Size	257 (bytes)
Received Time	2014-08-05 01:55:25
Included In Blocks	314018 (2014-08-05 02:00:40 +5 minutes)
Confirmations	9 Confirmations
Relayed by IP 	Blockchain.info
Visualize	View Tree Chart

Inputs and Outputs

Total Input	4.4775 BTC
Total Output	4.4774 BTC
Fees	0.0001 BTC
Estimated BTC Transacted	1.0194 BTC
Scripts	Show scripts & coinbase

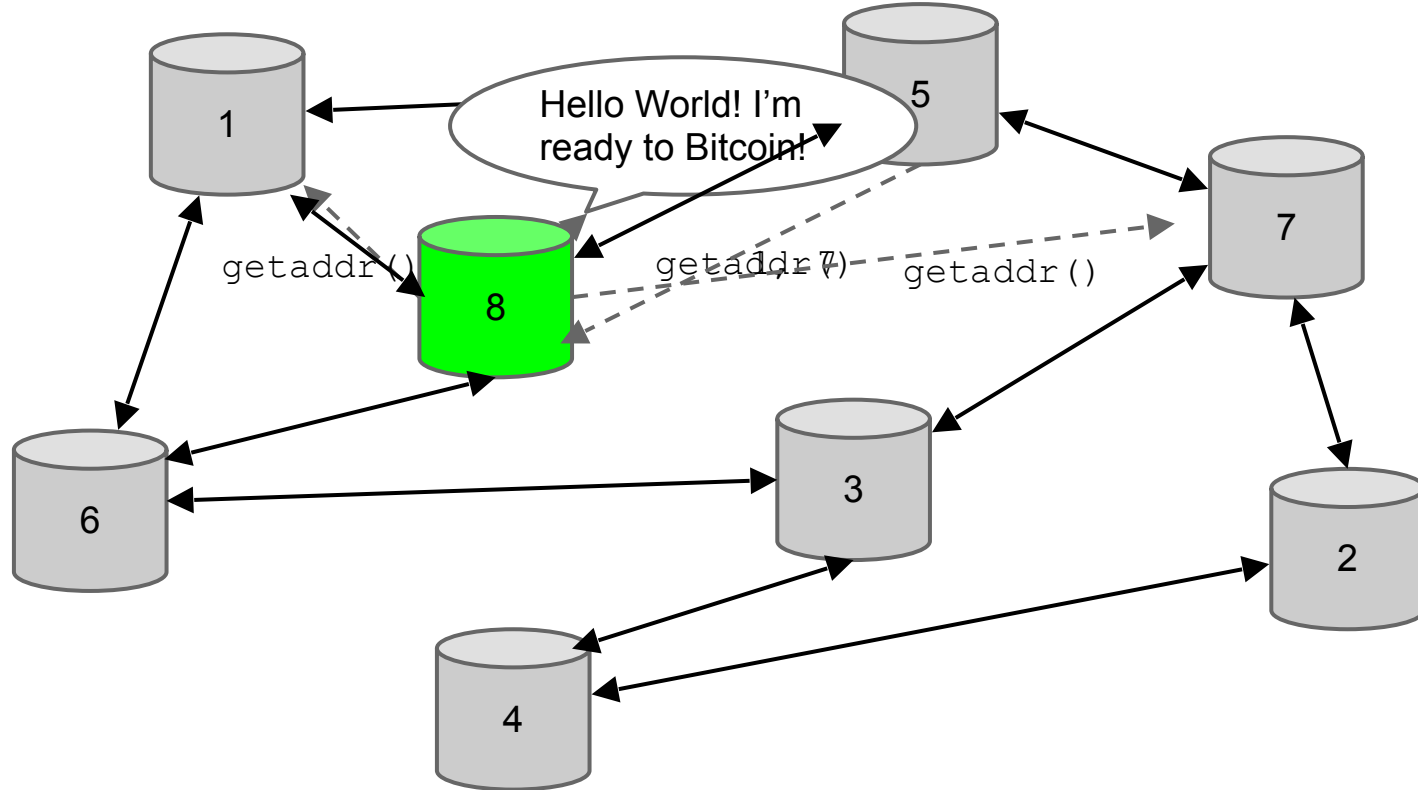
blockchain.info (and many other sites)

The Bitcoin network

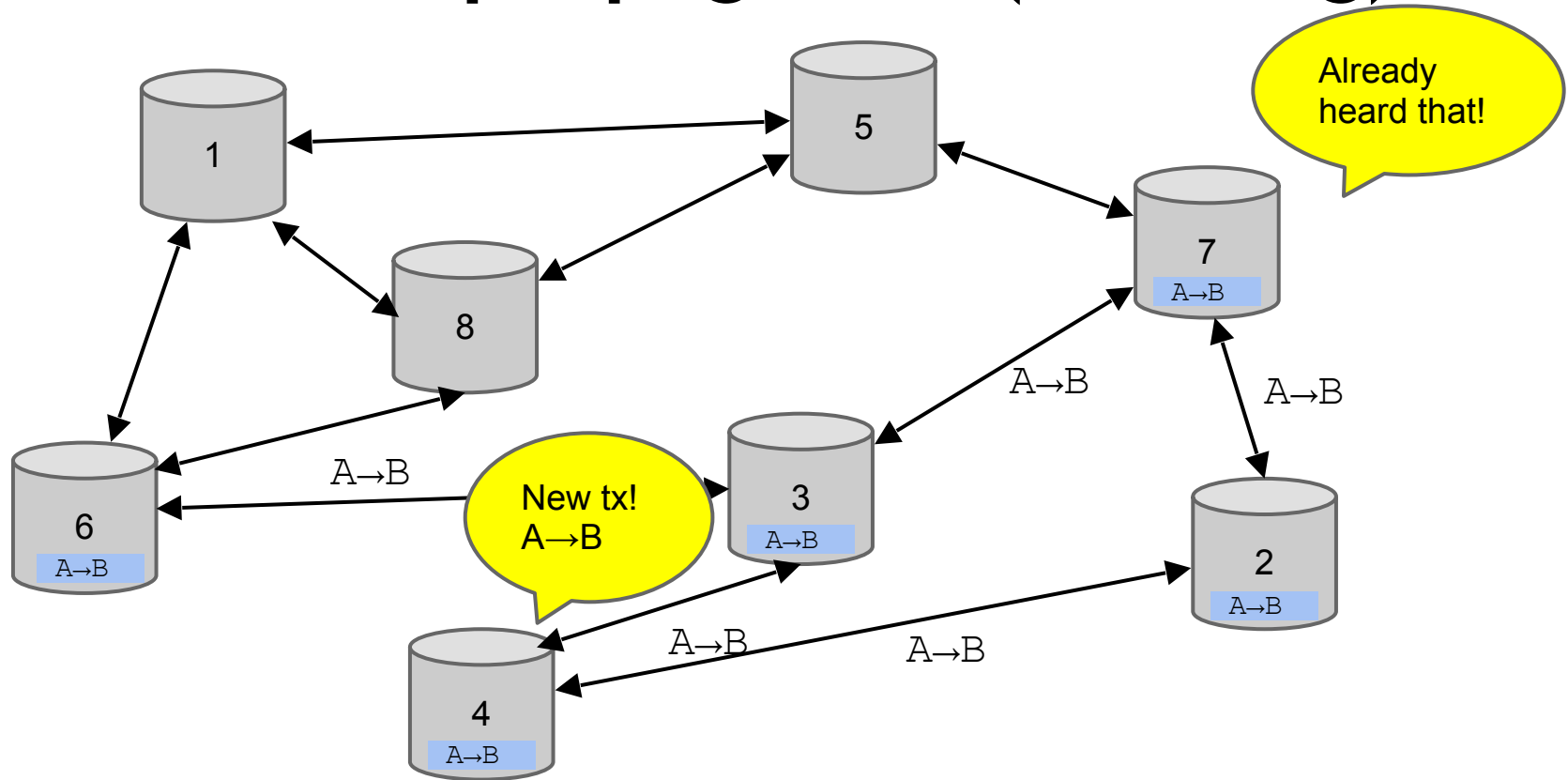
Bitcoin P2P network

- Ad-hoc protocol (runs on TCP port 8333)
- Ad-hoc network with random topology
- All nodes are equal
- New nodes can join at any time
- Forget non-responding nodes after 3 hr

Joining the Bitcoin P2P network



Transaction propagation (flooding)

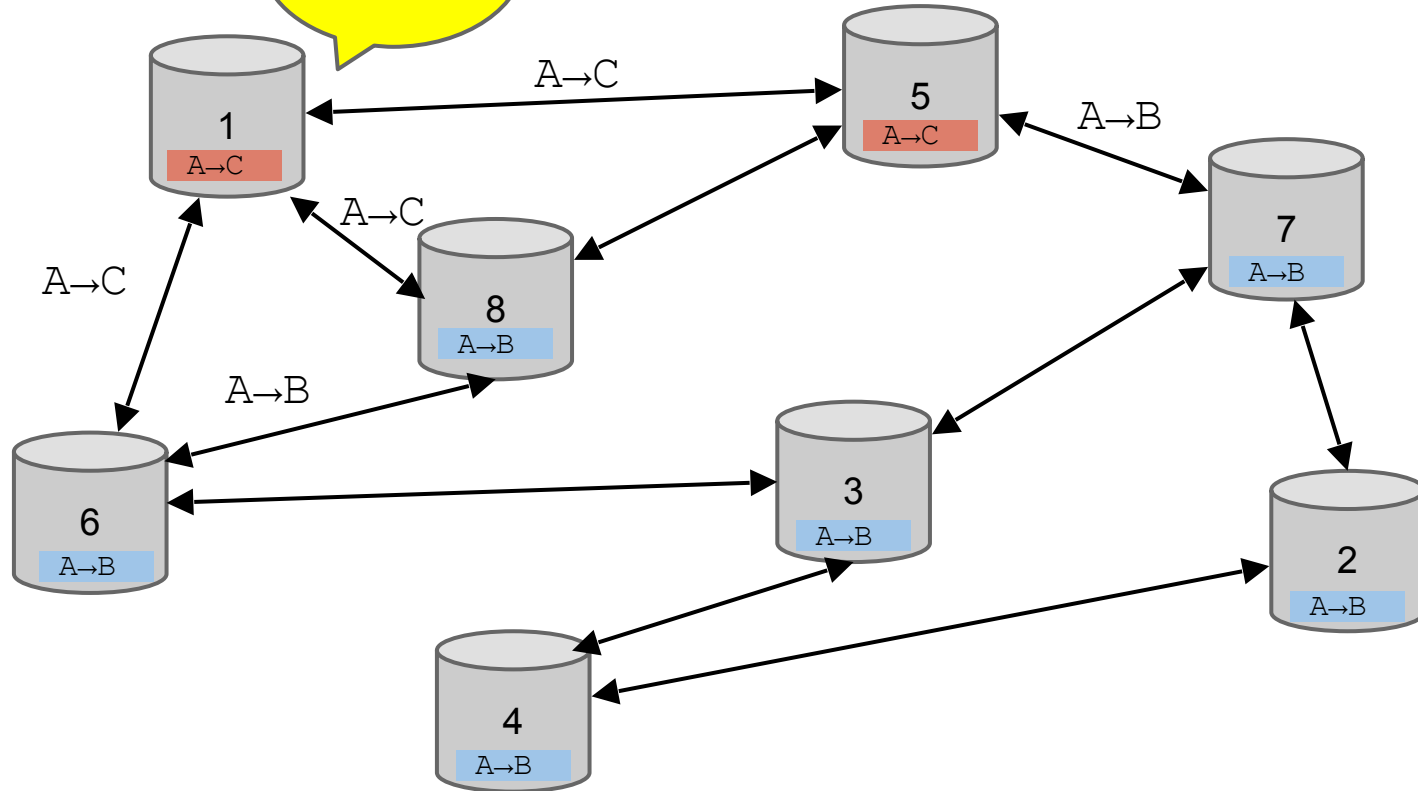


Should I relay a proposed transaction?

- Transaction valid with current block chain
 - (default) script matches a whitelist
 - Avoid unusual scripts
 - Haven't seen before
 - Avoid infinite loops
 - Doesn't conflict with others I've relayed
 - Avoid double-spends
- 
- The diagram consists of three grey arrows pointing from a green box to specific items in the list. The first arrow points from the green box to 'Avoid unusual scripts'. The second arrow points from the green box to 'Avoid infinite loops'. The third arrow points from the green box to 'Avoid double-spends'.

Sanity checks only...
Some nodes may ignore them!

Nodes differ on transaction pool



Race conditions

Transactions or blocks may *conflict*

- Default behavior: accept what you hear first
- Network position matters
- Miners may implement other logic!

Block propagation nearly identical

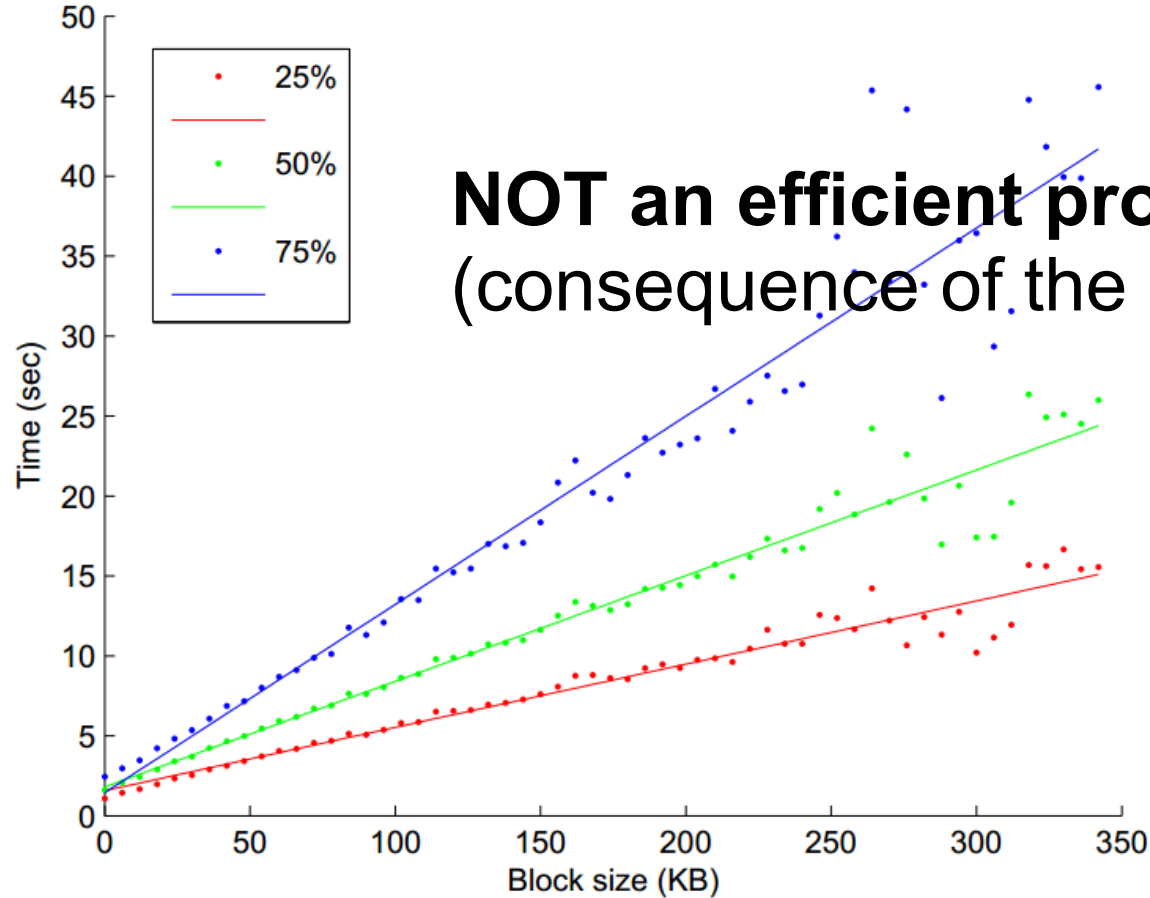
Relay a new block when you hear it if:

- Block meets the hash target
- Block has all valid transactions
 - Run *all* scripts, even if you wouldn't relay
- Block builds on current longest chain
 - Avoid forks



Sanity check
Also may be ignored...

Block Propagation Times



NOT an efficient protocol
(consequence of the design)

How big is the network?

- Unclear how to measure exactly
- Estimates-up to 1M IP addresses/month
- Only about 25K nodes reachable
 - Permanently connected
 - Fully-validate

Fully-validating nodes

- Permanently connected
- Store entire block chain
- Hear and forward every node/transaction

Thin/SPV clients (not fully-validating)

Idea: don't store everything

- Store block headers only
- Request transactions as needed
 - To verify incoming payment
- Trust fully-validating nodes

Limitations & improvements

Hard-coded limits in Bitcoin

- 10 min. average creation time per block
- 1 M bytes in a block (obsolete)
- 21M total bitcoins maximum
- 50,25,12.5... bitcoin mining reward

} These affect
economic
balance of
power too much
to change now

Throughput limits in Bitcoin

- 1 M bytes/block (10 min)
- >250 bytes/transaction
- 7 transactions/block

Improving throughput: A strong
motivation for Altcoins

Comparison

- VISA: 2,000-10,000 transactions/sec
- PayPal: 50-100 transaction/sec

Cryptographic limits in Bitcoin

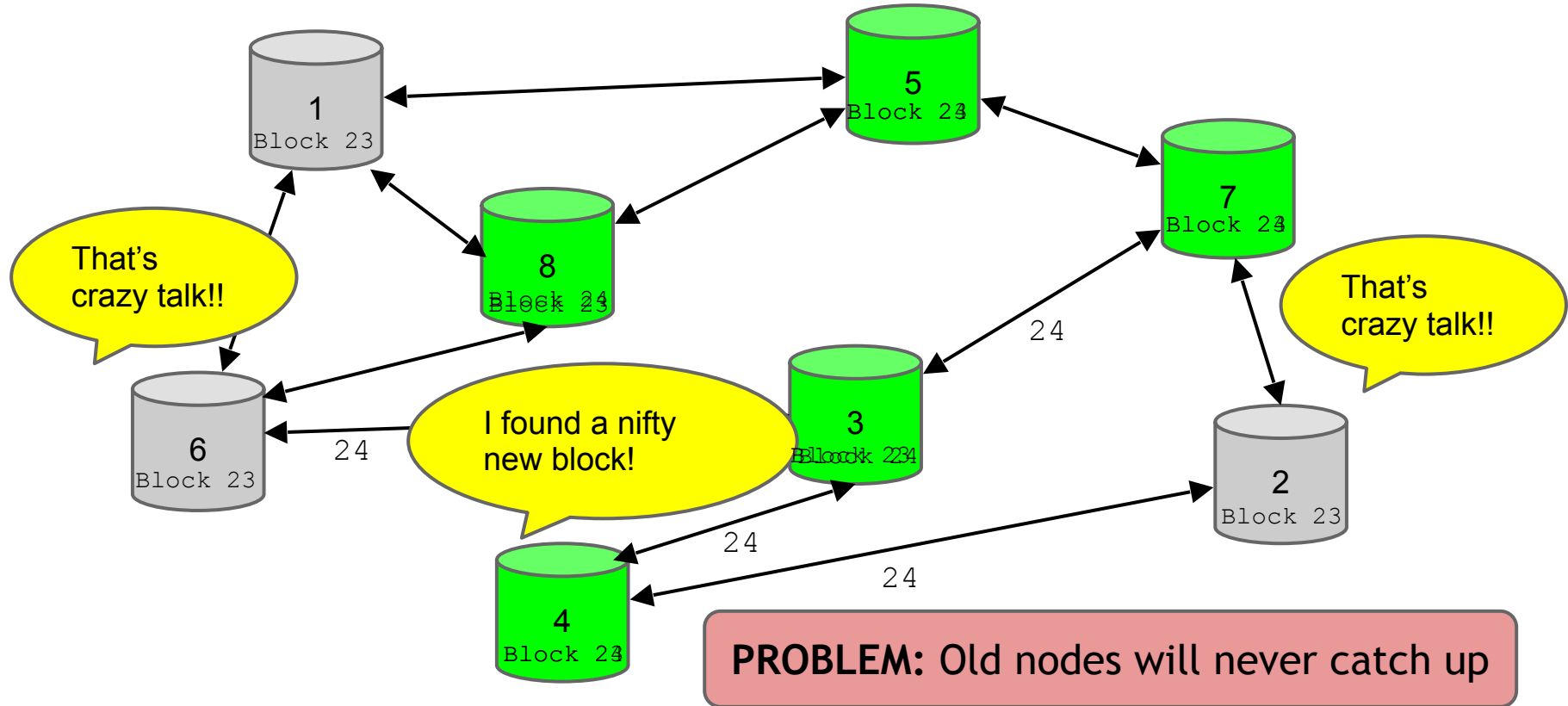
- Only 1 signature algorithm (ECDSA/P256)
- Hard-coded hash functions

Some of these crypto primitives used here might break by 2040 (e.g., collision-found in hash function, or powerful quantum computer breaks ECDSA)...

Why not update Bitcoin software to overcome these limitations?

- Many of these changes require “hard forks”, which are currently considered unacceptable

“Hard-forking” changes to Bitcoin



Soft forks

Observation: we can add new features which only *limit* the set of valid transactions

Need majority of nodes to enforce new rules

Old nodes will approve

RISK: Old nodes might mine now-invalid blocks

Soft fork example: pay to script hash

<signature>
<<pubkey> OP_CHECKSIG>

OP_HASH160
<hash of redemption script>
OP_EQUAL

Old nodes will just approve the hash, not run the embedded script

Soft fork possibilities

- New signature schemes
- Extra per-block metadata
 - Shove in the coinbase parameter
 - Commit to unspent transaction tree in each block

Hard forks

- New op codes
- Changes to size limits
- Changes to mining rate
- Many small bug fixes

Currently seem unlikely to happen

Many of these issues addressed by Altcoins